Code Explanation

Data Transformation Module Explanation

The provided code is a data transformation module for an ETL (Extract, Transform, Load) pipeline, written in Python using PySpark. It contains three main functions: `clean_and_transform_data`, `enrich_sales_data`, and `add_sales_metrics`, which are used to clean, enrich, and add metrics to sales data.

Overview
This module takes in raw sales data and other relevant dataframes, cleans and transforms the data, enriches it with additional information, and adds advanced sales metrics. The output is a transformed and enriched DataFrame with additional insights.

Step 1: Cleaning and Transforming Sales Data
The `clean_and_transform_data` function takes in a raw sales DataFrame and returns a cleaned and transformed DataFrame. It handles missing values, drops rows with missing critical fields, and adds calculated columns.

```
cleaned_df = sales_df.na.fill(0, ["quantity", "unit_price"])
cleaned_df = cleaned_df.dropna(subset=["sale_id", "product_id", "dat
e"])
transformed_df = cleaned_df.withColumn(
    "total_amount", F.col("quantity") * F.col("unit_price")
).withColumn(
    "year", F.year(F.to_date(F.col("date")))
).withColumn(
    "month", F.month(F.to_date(F.col("date")))
).withColumn(
    "day", F.dayofmonth(F.to_date(F.col("date")))
)
```

Step 2: Enriching Sales Data
The `enrich_sales_data` function takes in sales, products, and customers DataFrames and returns an enriched sales DataFrame. It joins the sales data with product and customer information.

```
enriched_df = sales_df.join(
    products_df.select("product_id", "product_name", "category"),
    on="product_id",
    how="left"
)
enriched_df = enriched_df.join(
    customers_df.select("customer_id", "customer_name", "region"),
    on="customer_id",
    how="left"
)
```

Step 3: Adding Advanced Sales Metrics
The `add_sales_metrics` function takes in a sales DataFrame and returns a DataFrame with additional metrics. It defines window specifications and adds metrics such as total sales by product and customer, and daily region rank.

```
window_by_product = Window.partitionBy("product_id")
window_by_customer = Window.partitionBy("customer_id")
window_by_date_region = Window.partitionBy("date", "region")

metrics_df = sales_df.withColumn(
    "product_total_sales", F.sum("total_amount").over(window_by_prod
uct)
).withColumn(
    "customer_total_sales", F.sum("total_amount").over(window_by_cus
tomer)
).withColumn(
    "daily_region_rank", F.rank().over(
        window_by_date_region.orderBy(F.desc("total_amount"))
    )
)
```